

# DOCUMENTATION TECHNIQUE

## Application Desktop Prestalia

Plateforme de mise en relation prestataires / clients

**Application WinUI 3 (Windows App SDK)**

### Équipe projet

Nathan • Jude • Kelly

Version 1.0 — Juin 2026

# 1. Présentation du projet

Prestalia est une application desktop développée sous Windows avec WinUI 3 (Windows App SDK). Elle permet à un administrateur de gérer la plateforme de mise en relation entre prestataires de services et clients : validation des comptes prestataires, gestion des catégories et supervision globale de l'activité.

## 1.1 Objectifs

- Fournir une interface d'administration native Windows, rapide et réactive
- Permettre la modération des prestataires (approbation / rejet)
- Gérer le référentiel de catégories de prestations
- Superviser l'activité de la plateforme via un tableau de bord en temps réel
- Assurer une expérience fluide via une navigation sidebar persistante

## 1.2 Informations générales

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| Nom du projet      | Prestalia — Admin Desktop                               |
| Type d'application | Application desktop Windows (WinUI 3 / Windows App SDK) |
| Langage principal  | C# (.NET 8)                                             |
| Framework UI       | WinUI 3 (Windows App SDK 1.x)                           |
| Plateforme cible   | Windows 10 (1809+) / Windows 11                         |
| Architecture       | MVVM (Model-View-ViewModel)                             |
| Équipe             | Nathan, Jude, Kelly                                     |
| Version            | 1.0.0                                                   |
| Date de livraison  | Juin 2026                                               |

# 2. Architecture technique

L'application repose sur le patron d'architecture MVVM, qui découple clairement la logique métier de la couche de présentation. Ce choix facilite la testabilité du code, la maintenabilité à long terme et la parallélisation du développement entre les membres de l'équipe.

## 2.1 Structure des couches

| Couche              | Rôle                                  | Technologies  |
|---------------------|---------------------------------------|---------------|
| Présentation (View) | Pages XAML, composants UI, navigation | WinUI 3, XAML |

|                      |                                                         |                                            |
|----------------------|---------------------------------------------------------|--------------------------------------------|
| ViewModel            | Logique de présentation, liaison de données, commandes  | C#, INotifyPropertyChanged, RelayCommand   |
| Modèle (Model)       | Entités métier (Prestataire, Categorie, Utilisateur...) | C# POCO classes                            |
| Service / Repository | Accès aux données, appels API REST                      | HttpClient, JSON, System.Text.Json         |
| Infrastructure       | Configuration, injection de dépendances, navigation     | Microsoft.Extensions.DI, NavigationService |

## 2.2 Arborescence du projet

```
Prestalia.Admin/
├── App.xaml / App.xaml.cs      - Point d'entrée, DI container
├── Assets/                     - Images, icônes, logo
├── Models/
│   ├── Prestataire.cs
│   ├── Categorie.cs
│   ├── Utilisateur.cs
│   └── Reservation.cs
├── ViewModels/
│   ├── DashboardViewModel.cs
│   ├── CategoriesViewModel.cs
│   ├── ActivitesViewModel.cs
│   └── LoginViewModel.cs
├── Views/
│   ├── LoginPage.xaml
│   ├── DashboardPage.xaml
│   ├── CategoriesPage.xaml
│   └── ActivitesPage.xaml
├── Services/
│   ├── ApiService.cs          - HTTP REST client
│   ├── AuthService.cs         - Authentification, JWT
│   └── NavigationService.cs
└── Helpers/
    ├── RelayCommand.cs
    └── ObservableObject.cs
```

## 2.3 Navigation

La navigation est assurée par un NavigationView (sidebar) persistant sur la gauche de la fenêtre. Quatre destinations sont accessibles :

| Entrée sidebar | Page           | Description                                              |
|----------------|----------------|----------------------------------------------------------|
| Accueil        | DashboardPage  | Tableau de bord avec compteurs et liste des prestataires |
| Catégories     | CategoriesPage | CRUD des catégories de prestations                       |
| Activités      | ActivitesPage  | Journal d'activité de la plateforme (en cours)           |
| Paramètres     | SettingsPage   | Configuration de l'application                           |

### 3. Fonctionnalités de l'application

#### 3.1 Authentification

L'écran de connexion est la première interface affichée au démarrage. Elle comporte :

- Un champ email (test@example.com) avec validation de format
- Un champ mot de passe masqué
- Un bouton « Se connecter » déclenchant l'appel à l'API d'authentification
- La gestion des erreurs (identifiants invalides, serveur indisponible)

Après authentification réussie, un token JWT est stocké en mémoire et injecté dans chaque requête HTTP suivante via un DelegatingHandler.

#### 3.2 Tableau de bord administrateur

La page d'accueil synthétise l'état de la plateforme en quatre compteurs :

| Indicateur             | Valeur affichée       | Description                           |
|------------------------|-----------------------|---------------------------------------|
| Total des prestataires | Nombre entier         | Tous statuts confondus                |
| En attente             | Nombre + badge orange | Prestataires nécessitant une action   |
| Approuvés              | Nombre + icône ✓      | Prestataires actifs sur la plateforme |
| Rejetés                | Nombre + icône ✗      | Prestataires refusés                  |

Le tableau « Gestion des prestataires » liste l'ensemble des prestataires avec les colonnes suivantes : Prestataire, Catégorie, Ville, Note, Statut et Actions. Des filtres rapides permettent de basculer entre les vues Tous / En attente / Approuvés / Rejetés. Des options de tri (par nom, date, note) et d'ordre (croissant / décroissant) complètent l'interface.

#### 3.3 Gestion des catégories

Cette page donne accès au référentiel des catégories de prestations. Fonctionnalités disponibles :

- Affichage en liste avec colonnes : Nom, Nombre de prestataires, Nombre de prestations, Date de création, Actions
- Ajout d'une nouvelle catégorie via le bouton « Ajouter » (formulaire en dialogue)
- Modification d'une catégorie existante
- Suppression d'une catégorie (avec confirmation)
- Tri et filtrage par nom ou date

#### 3.4 Actions sur un prestataire

Depuis le tableau de bord, l'administrateur peut effectuer les actions suivantes sur chaque prestataire :

- Voir le profil complet (détails, certificats, avis)

- Approuver un compte en attente → statut passe à « Approuvé »
- Rejeter un compte en attente → statut passe à « Rejeté »
- Suspendre un prestataire approuvé

## 4. Modèle de données

Le modèle conceptuel de données (MCD) de Prestalia s'articule autour de six entités principales. Voici la description de chacune avec ses attributs.

### 4.1 Entités principales

| Entité       | Clé primaire   | Attributs principaux                                                          |
|--------------|----------------|-------------------------------------------------------------------------------|
| Prestataires | Id_Prestataire | Nom, Téléphone, Description, Adresse, Expérience, Tarif, Statut, Id_Categorie |
| Utilisateurs | Id_Utilisateur | Nom, Prénom, Email, Mot_de_passe, Rôle                                        |
| Categories   | Id_Categorie   | Nom, Logo                                                                     |
| Reservations | Id_Reservation | Prix, Date_reservation, Description, Id_Prestataire, Id_Utilisateur           |
| Certificats  | Id_Certificats | Nom, URL, Statut, Id_Prestataire                                              |
| Commenter    | —              | Note, Commentaire, Id_Prestataire, Id_Utilisateur                             |

### 4.2 Relations

- Un Prestataire appartient à une Categorie (N,1)
- Un Prestataire possède plusieurs Certificats (1,N)
- Un Utilisateur peut faire plusieurs Reservations (1,N)
- Une Reservation est reçue par un seul Prestataire (N,1)
- Un Utilisateur peut commenter plusieurs Prestataires (N,N via Commenter)

## 5. Communication avec l'API REST

L'application desktop consomme une API REST pour toutes ses opérations. Le service ApiService.cs centralise les appels HTTP et gère la sérialisation JSON.

### 5.1 Endpoints utilisés (administration)

| Méthode | Endpoint | Description |
|---------|----------|-------------|
|---------|----------|-------------|

|        |                               |                                        |
|--------|-------------------------------|----------------------------------------|
| POST   | /api/auth/login               | Authentification, retourne un JWT      |
| GET    | /api/prestataires             | Liste de tous les prestataires         |
| GET    | /api/prestataires/{id}        | Détail d'un prestataire                |
| PATCH  | /api/prestataires/{id}/statut | Modifier le statut (approved/rejected) |
| GET    | /api/categories               | Liste des catégories                   |
| POST   | /api/categories               | Créer une catégorie                    |
| PUT    | /api/categories/{id}          | Modifier une catégorie                 |
| DELETE | /api/categories/{id}          | Supprimer une catégorie                |
| GET    | /api/reservations             | Liste des réservations                 |

## 5.2 Gestion des erreurs

- 401 Unauthorized → redirection vers la page de connexion
- 403 Forbidden → message d'accès refusé
- 404 Not Found → notification d'élément introuvable
- 500 Server Error → message d'erreur générique + retry automatique
- Timeout réseau → affichage d'une bannière d'erreur non-bloquante

## 6. Sécurité

### 6.1 Authentification JWT

Le token JWT est obtenu lors de la connexion et stocké en mémoire (non persisté sur disque). Il est injecté automatiquement dans le header Authorization de chaque requête HTTP :

```
// DelegatingHandler - injection automatique du token
protected override async Task<HttpResponseMessage> SendAsync(
    HttpRequestMessage request, CancellationToken cancellationToken)
{
    var token = _authService.GetToken();
    if (token != null)
        request.Headers.Authorization =
            new AuthenticationHeaderValue("Bearer", token);
    return await base.SendAsync(request, cancellationToken);
}
```

### 6.2 Contrôle d'accès

- Seuls les comptes avec le rôle « admin » peuvent accéder à l'application
- Le rôle est vérifié côté serveur à chaque requête
- Les actions sensibles (approbation, suppression) nécessitent une confirmation utilisateur

## 7. Gestion de projet

Le projet Prestalia a été géré selon une méthodologie Agile, structurée en quatre sprints successifs.

### 7.1 Organisation des sprints

| Sprint   | Thème                       | Livrables                                                    |
|----------|-----------------------------|--------------------------------------------------------------|
| Sprint 1 | Conception                  | Maquettes, MCD/MLD, diagrammes UML, charte graphique         |
| Sprint 2 | Développement               | Modèles, ViewModels, pages XAML, intégration API             |
| Sprint 3 | Tests                       | Tests fonctionnels, tests de performance, correction de bugs |
| Sprint 4 | Déploiement & améliorations | Package MSIX, mise en production, retours utilisateurs       |

### 7.2 Répartition des rôles

| Membre | Rôle principal | Contributions                                                     |
|--------|----------------|-------------------------------------------------------------------|
| Nathan | Lead Developer | Architecture MVVM, ApiService, DashboardViewModel, navigation     |
| Jude   | Frontend / UX  | Pages XAML, charte graphique, composants UI, animations           |
| Kelly  | Backend / QA   | Modèles de données, diagrammes, tests fonctionnels, documentation |

## 8. Installation et déploiement

### 8.1 Prérequis

- Windows 10 version 1809 (build 17763) ou Windows 11
- .NET 8 Runtime installé
- Windows App SDK 1.x (redistribuable inclus dans le package)
- Connexion réseau pour les appels API

### 8.2 Déploiement via MSIX

L'application est distribuée sous forme de package MSIX signé. La procédure d'installation :

- Télécharger le fichier Prestalia.Admin\_1.0.0\_x64.msix
- Double-cliquer sur le fichier pour lancer le programme d'installation
- Accepter l'installation si le certificat de signature est approuvé

- L'application apparaît dans le menu Démarrer sous « Prestalia »

### 8.3 Configuration

```
// appsettings.json - configuration de l'environnement
{
  "ApiBaseUrl": "https://api.prestalia.fr/v1",
  "Timeout": 30,
  "Environment": "Production"
}
```

## Annexe — Glossaire

| Terme       | Définition                                                                   |
|-------------|------------------------------------------------------------------------------|
| Prestataire | Professionnel proposant ses services sur la plateforme                       |
| Statut      | État du compte prestataire : En attente / Approuvé / Rejeté                  |
| JWT         | JSON Web Token — mécanisme d'authentification sans état                      |
| MVVM        | Model-View-ViewModel — patron d'architecture UI                              |
| WinUI 3     | Framework UI natif Windows basé sur le Windows App SDK                       |
| MSIX        | Format de packaging moderne pour les applications Windows                    |
| DXA         | Device-independent eXtra-small units — unité de mesure Word (1440 = 1 pouce) |